Final Project Methodology — Forecasting Philippine Pump Prices

Project: Predicting weekly NCR pump prices using crude oil benchmarks, FX, and macro features **Course coverage:** DMW (data wrangling/scraping/storage/PCA-SVD), ML (supervised models), DVS (visualization) **Sample period:** January 2017 – present **Target:** Diesel, RON 91, RON 95, RON 97 (and optionally Diesel Plus, RON 100, Kerosene) **Forecast horizons:** 1, 2, and 4 weeks ahead

---

0. The economic mechanism you are modeling — read this first

Before touching any data, internalize the Philippine pump-price formula. It will dictate every feature you collect and explain every modeling choice you make.

In the Philippines, pump prices are NOT set by free-market auction at the station. They follow a **replacement-cost formula** that the oil companies and DOE agreed on:

```
PumpPrice_t = (MOPS_{t-1, weekly avg} × USD_PHP_{t-1, weekly avg})
            + Freight + Insurance
            + Excise tax (₱10/L gasoline, ₱6/L diesel, ₱5/L kerosene as of TRAIN)
            + 12% VAT (applied on top)
            + Distributor / dealer margin
```

Key facts that shape the methodology:

1. **MOPS = Mean of Platts Singapore** is the regional refined-product spot benchmark. PH pump prices are anchored to MOPS, *not* to Brent/WTI directly. Brent/WTI is one step further upstream (crude oil), MOPS is the refined product. There is a refining margin (the "crack spread") between them.
2. **The Philippines uses Dubai crude as the relevant benchmark, not Brent** — Asian-Pacific refiners price off Dubai/Oman because of geographic logistics and sulfur content. Brent is used in MP1 because it's freely available via EIA, but **Dubai is closer to the truth**. World Bank Pink Sheet publishes monthly Dubai prices for free; daily Dubai is paywalled but the monthly is enough as a robustness check.
3. **The lag is one week, not zero.** Oil companies announce on Monday based on the *previous week's* MOPS+FX averages, and prices change Tuesday 6 AM. So the "right" feature for predicting pump price in week T is the MOPS+FX average over week T-1. Using week-T contemporaneous values would be data leakage by mechanism.
4. **Excise taxes are step functions, not continuous.** TRAIN law (RA 10963) raised them in 3 tranches Jan 2018, Jan 2019, Jan 2020. RA 12316 (March 2026) gives the President

temporary suspension power on LPG/kerosene when Dubai > $80. These are **dummy variable** events, not noise.

5. **The DOE prescribes a *minimum* price reduction** during rollback weeks; oil companies can cut more but not less. So pump-price moves are right-bounded on the downside but unbounded on the upside — built-in asymmetry. This connects directly to the **Rockets and Feathers Hypothesis** (covered in the modeling section).

If your model recovers something close to the formula above with sensible coefficients, you've validated it economically. If your "best" model has Brent at lag-3 with weight 0.8 and FX with weight 0.01, something is wrong.

---

1. Data sources — what to collect, why, and from where

Each row below is a feature group with: economic motivation → exact source → cadence → cost.

1.1. Required (the core feature set)

#	Feature	Why it matters	Source	Cadence	Cost
F1	**MOPS Gasoil 10ppm** (diesel benchmark)	The actual input in the PH pricing formula for diesel	S&P Platts (paywalled). **Free proxy: Singapore gasoil futures via investing.com** or the EIA series "Singapore Gasoil/Diesel Spot FOB"	Daily	Free (proxy
F2	**MOPS Mogas 92/95** (gasoline benchmarks)	The actual input for RON 91/95	Same — Singapore gasoline FOB Platts proxy via investing.com	Daily	Free (proxy
F3	**Brent crude spot**	Crude proxy when MOPS unavailable; you already have it	EIA `PET.RBRTE.D` (you have this in `Brent_WTI_Prices_2020_to_Present.csv`)	Daily	Free
F4	**WTI crude spot**	Robustness; you already have it. Will likely drop after EDA.	EIA `PET.RWTC.D`	Daily	Free

#	Feature	Why it matters	Source	Cadence	Cost
F5	**Dubai crude spot**	The "correct" benchmark for Asian refiners; key robustness check	World Bank Pink Sheet `KCRUDE_D` (monthly only, free); FRED `POILDUBUSDQ` (quarterly); daily is paywalled	Monthly free	Free
F6	**USD/PHP exchange rate**	Direct multiplicative term in the formula. Critical.	BSP Reference Exchange Rate Bulletin (daily); also yfinance `PHP=X` or investing.com	Daily	Free
F7	**DOE NCR pump prices** (target)	The target variable	DOE Oil Monitor PDFs (you scraped 2020-12 to present; need to extend to 2017)	Weekly	Free

1.2. Strongly recommended (improves model meaningfully)

#	Feature	Why	Source
F8	**Excise tax level**	Step changes Jan 2018/2019/2020; RA 12316 suspensions	Manual lookup; encode as `excise_diesel_php_per_L`, `excise_gasoline_php_per_L`, `excise_kerosene_php_per_L`, recorded with effective date
F9	**VAT rate**	Constant 12% over the sample, but document it for completeness	National Internal Revenue Code
F10	**Holiday / monitoring schedule**	Adjustments don't always land Tuesday in a holiday week	PH official holidays calendar (free CSV widely available)
F11	**Brent realized volatility (rolling)**	Captures regime; rocket-and-feather effect is volatility-conditional	Computed from F3

#	Feature	Why	Source
F12	**MOPS-Brent crack spread**	The refining margin; isolates refining-cost dynamics from crude dynamics	F1 – F3

1.3. Optional / experimental (try if time permits, can be in "future work")

#	Feature	Why	Source
F13	**Headline / core CPI**	Macro context; CPI affects margins and dealer behavior	PSA OpenSTAT (monthly)
F14	**BSP policy rate**	Macro context; rare but discrete moves	BSP website (event-based)
F15	**Fuel subsidy / Pantawid Pasada flags**	Periodic government interventions distort the formula	DOE press releases — encode as binary periods
F16	**Geopolitical / event dummies**	Russia–Ukraine 2022, Strait of Hormuz tensions 2025–26, OPEC+ cuts. Lunor et al. (2023) and Wen et al. (2025) both used these.	News-based; encode as binary windows
F17	**EMV Infectious Disease Tracker**	Used by Lunor et al. (2023) and Tuna & Tuna (2022) — proxies pandemic risk to oil markets	FRED `INFECTDISEMVTRACKD` (daily)
F18	**Number of new global COVID cases**	Same justification	WHO COVID-19 dashboard (daily)
F19	**Russian ruble / Ukrainian hryvnia FX**	Firouzjaee & Khaliliyan (2022) found the correlation to oil shifted during the conflict	investing.com
F20	**Google Trends "gas price" PH**	Demand-side sentiment proxy	Google Trends (free)

1.4. Why I'm pushing you toward MOPS even though it's harder to get

The He (2023) paper you uploaded ("Forecasting Gasoline Price with time Series Models") gives

the cleanest empirical demonstration of why this matters. He shows:

- LM3 (regression on **only** crude oil price) → $R^2 = 0.874$, test RMSE = 0.739 — "fair"
- LM1 (trend + seasonality + GDP + CPI + crude) → $R^2 = 0.947$, test RMSE = 0.208 — much better

His punchline: *"crude oil price plays a major role (55%) in determining the gasoline price, but the crude oil price alone is not sufficient."* Exactly. For the PH, the missing 45% is captured by **MOPS spread + USD/PHP + excise + VAT + margin**. Skip these and you're capping your test R^2 around 0.85 no matter how fancy the model.

1.5. Going back to 2017 — the data gap you need to close

Your current `DOE_NCR_OilPrices_Compiled.csv` starts **2020-12-22**. To get back to **2017-01**, you need to re-scrape DOE Oil Monitor archives. Two routes:

- **Primary**: Replay the DOE PDF scraper from MP1 against archive URLs going back to 2017. The DOE filename schema changed multiple times; you already solved this for 2020+, just extend the date list. Filenames typically follow `oil_monitor_YYYY-MM-DD.pdf` with variants — your existing parser logic should generalize. Budget 1 working day for this.
- **Backup**: Web Archive (web.archive.org) snapshots of DOE Oil Monitor, which preserve the historical PDFs even if DOE moves them.

Practical: If you cannot recover clean 2017–2020 DOE data within 2 days of effort, drop the start date to **Jan 2018** (right after the first TRAIN excise hike, so your sample has consistent post-TRAIN tax structure and you avoid the messy 2017 transition). 2018-01 to 2026-04 is still ~430 weeks — plenty for ML.

---

2. The DMW components — addressing rubric requirements directly

The DMW final project rubric requires:

1. Data acquisition via web scraping AND web API (≥3 sources, mix of methods)
2. Storage in structured + semi-structured + SQL formats
3. Feature reduction via PCA or SVD
4. Modular code, environment.yml, unit tests, develop branch + PR workflow
5. Final project report

Map your sources to this:

2.1. Three+ sources, mixed acquisition methods

Source	Method	Format produced
EIA (Brent, WTI)	**REST API** (api.eia.gov, free with key)	JSON pulled, parsed, persisted to Parquet
BSP Reference Exchange Rate	**Web scraping** (BeautifulSoup; HTML tables)	HTML scraped → CSV (structured) + raw HTML cached
DOE Oil Monitor	**PDF scraping** (existing pipeline, extended to 2017)	PDF parsed → CSV (structured)
World Bank Pink Sheet	**REST API or XLSX download**	XLSX → Parquet
Singapore Gasoil/Mogas (MOPS proxy)	**Web scraping** (investing.com or barchart) OR yfinance API	CSV / Parquet

That's 5 sources, with 2 web scraping + 2 API + 1 PDF parse — exceeds the minimum and addresses the "scraping headers should include contact email" comment by setting `User-Agent` properly.

2.2. Storage formats — addressing the semi-structured deduction

- **Structured (non-SQL)**: Parquet files for cleaned series. Parquet is columnar, type-preserving, and the modern choice.
- **Semi-structured**: Keep raw API responses as **JSON** in `data/raw/eia/` and `data/raw/wb/`. **Do NOT delete them after parsing.** They're your audit trail and they satisfy the rubric. Also store DOE PDFs themselves in `data/raw/doe/`.
- **SQL**: SQLite database `pump_prices.db` with normalized schema (see §6 for schema).

2.3. PCA/SVD as required dimensionality reduction

Two natural places for PCA:

1. **Lag-feature compression**: After you build 8 weekly lags × ~6 base features = ~48 columns, many are correlated (Brent_lag1 vs Brent_lag2 are 0.95+ correlated). Run PCA on the lag-feature block, keep components explaining 95% of variance. Document the variance explained and use the components as model inputs alongside the non-lagged features. This is exactly what "reduce data noise and variable intercorrelation" looks like in practice and matches what Lunor et al. (2023) did for their Philippine pump-price ML study (PCA before MLR/SVR/RFR).

2. **Cross-product price-level decomposition**: PCA on the 7 pump-product time series (Diesel, RON 91/95/97/100, Diesel Plus, Kerosene). The first PC will be "the level of fuel prices in NCR" and the second will be "diesel-vs-gasoline spread" or similar. Useful for the report's EDA section and addresses the DVS course as well.

---

3. Modeling methodology — the supervised ML pipeline

This section maps directly to the Notebooks 2 (preprocessing), 3A (linear), 3B (classification), and 4 (trees) you've used in ML class.

3.1. Target framing — three options, pick one as primary

Option	Target	When to use
A. Price level $\hat{y}$ = `pump_t+h`	Easiest to interpret; matches business need ("what will diesel cost in 4 weeks?")	**Primary recommendation**
B. Log-return	$\hat{y}$ = `log(p_t+h / p_t)`	Standard in finance; stationary by construction; what Wen et al. (2025) use
C. Adjustment direction (3-class)	$\hat{y} \in$ {rollback, no change, hike}	Engages your ML Notebook 3B classification skills directly; matches DOE's weekly announcement cadence

Recommendation: Do **A as primary** (regression, the bulk of the work, MAE/RMSE/MAPE evaluation). Then add **C as a secondary classification analysis** in a section called "Predicting weekly price-change direction" — this lets you use logistic regression and SVM from Notebook 3B and tree classifiers from Notebook 4. It also directly mirrors how a real procurement team would consume the output ("should I fill up Monday evening?").

3.2. Train/test/validation split — walk-forward, never random

Standard time-series split with gap:

- Total weeks (2018–2026): ~430
- Initial training: 208 weeks (4 years, 2018–2021)
- Validation folds: rolling, each 26 weeks (6 months)
- Final held-out test: last 52 weeks (May 2025 – April 2026), **never touched until final model is locked**

`sklearn.model_selection.TimeSeriesSplit(n_splits=5, gap=h)` where `h` = forecast horizon. The gap is essential — it prevents leakage of the h-step-ahead target into training.

Random k-fold is forbidden. The Lunor et al. paper used 5 nested splits with 4-month testing windows and that's a clean reference for your write-up.

3.3. Feature preprocessing — concrete steps

For each weekly observation at week T:

1. **Build lag features**: For each of {MOPS_diesel, MOPS_gasoline, Brent, USD_PHP, pump_target_own_lag}, create lags 1, 2, 3, 4, 6, 8 weeks. Skip lag 0 — that's leakage given the 1-week formula lag.
2. **Build rolling features**: 4w mean, 4w std, 8w mean of Brent and USD_PHP. Captures regime.
3. **Build derived features**: `crack_spread = MOPS_diesel - Brent`, `MOPS_in_PHP = MOPS_diesel × USD_PHP` (directly mimics the formula). The latter is the single most important engineered feature you can build.
4. **Calendar features**: `month`, `iso_week`, `is_holiday_week_PH`, `weeks_since_excise_change`.
5. **Event dummies**: `event_covid_2020` (Mar–Sep 2020), `event_ru_ua_war` (Feb 2022 onward), `event_hormuz_2025` (Mar 2026 onward), `excise_suspension_active`.
6. **Standardize** numeric features (`StandardScaler` fit only on training fold) for linear/SVM models. Tree models don't need it.
7. **Drop rows with NaN** from initial lag construction.
8. **Apply PCA** to the lag-feature block (see §2.3) → keep components explaining 95% variance. Document the loadings.

3.4. Models — staged from baseline to advanced

This is the heart of the pipeline. **Beat each tier before moving to the next.**

Tier 0 — Naive baselines (you must beat these to claim ML adds value)

- **Persistence**: $\hat{y}_{t+h} = y_t$
- **Seasonal naive**: $\hat{y}_{t+h} = y_{t+h-52}$
- **MOPS-formula baseline** *(your novel baseline)*: $\hat{y}_{t+h} = \alpha + \beta \cdot (MOPS_{t-1+h} \times USD_PHP_{t-1+h})$ — encodes the actual pricing mechanism. If your ML can't beat this, you haven't actually shown ML adds value over economic structure.

Report MAE, RMSE, MAPE, MASE for all baselines.

Tier 1 — Linear models (ML Notebook 3A)

Justification: Both He (2023) and Eliwa et al. (2024) (the ANFIS paper) found that **linear regression with the right features beats almost everything**. He's LM1 hit $R^2 = 0.947$ with just trend+season+GDP+CPI+oil. Eliwa's Table 9 shows linear regression beat AdaBoost, ExtraTrees, RandomForest, and XGBoost on the same data. This is a recurring finding in oil-price ML — *the relationship is mostly linear once you have the right features*.

- **Multiple Linear Regression (OLS)** — interpretable baseline, gives you coefficient magnitudes
- **Ridge regression** — your features will be highly collinear (Brent_lag1, Brent_lag2 are nearly identical); L2 handles this
- **Lasso regression** — does feature selection for free; tells you which lags actually matter
- **ElasticNet** — combines ridge + lasso

Use `RidgeCV`, `LassoCV`, `ElasticNetCV` so the regularization strength is tuned per fold.

Tier 2 — SVM and tree-based models (ML Notebooks 3B, 4)

- **Support Vector Regression (SVR)** with RBF kernel. Hyperparameters C, gamma, epsilon tuned via RandomizedSearchCV over folds. SVR was the joint-best in Lunor et al.'s PH pump price study. Notebook 3B covers SVM directly.
- **Random Forest Regression**. Notebook 4. Tune `n_estimators`, `max_depth`, `min_samples_leaf`.
- **Gradient Boosted Trees (sklearn GradientBoostingRegressor)**. Notebook 4 (GBM section).
- **XGBoost / LightGBM**. Same family, modern implementations. Your prof explicitly suggested XGBoost-with-exogenous as "may be among the best." The Wen et al. (2025) paper confirms this: regularization (Ridge/Lasso/ElasticNet) and gradient boosting both work well for oil-product price forecasting at h=1 to h=8 weeks.

Tier 3 — Optional advanced (mention if time, otherwise future work)

- **kNN regression** (Notebook 1B) — useful as a non-parametric baseline, especially good for capturing regime-similar weeks
- **ANFIS** — Eliwa et al.'s paper achieves $R^2 = 0.997$ on US weekly gasoline using ANFIS with the previous price as a feature. Worth attempting if you have time, but warn yourself: their result is suspicious of overfitting (the previous-price feature alone with autocorrelation 0.95+ will give 0.99 R^2 with any model — see §4.2 below).

- **LSTM / GRU / N-BEATS** — Ljubić et al. (2023) used LSTM on Bosnian fuel prices and got Pearson 0.97. Possible, but **gradient boosting will likely match or beat LSTM at your sample size**. Do this only as a comparison curiosity.

3.5. Hyperparameter tuning

Use `RandomizedSearchCV` with the time-series split as the CV iterator. Budget:

- Ridge/Lasso/ElasticNet: 50 iterations
- SVR: 100 iterations (kernel ∈ {linear, rbf}, C ∈ log[0.01, 100], gamma ∈ log[0.001, 1], epsilon ∈ [0.01, 0.5])
- RF: 100 iterations
- XGBoost/LightGBM: 200 iterations

This matches the Lunor et al. setup (300–600 iterations across nested splits) and is what your prof would expect.

3.6. Evaluation metrics

Always report all of:

- **MAE** (₱/L) — most interpretable
- **RMSE** (₱/L) — standard, penalizes outliers
- **MAPE** (%) — standard in oil-price literature; lets you compare to published benchmarks (Lu et al. 2021 published a decade-review benchmark range of 0.131%–19.2% MAPE for oil price prediction; Lunor et al. got 3.13%–12.67%)
- **MASE** (Mean Absolute Scaled Error vs. naive) — MASE < 1 means you beat persistence
- **Diebold–Mariano test** — formal stat test that your model's forecast errors are significantly different from baseline. Use this when you claim "Model A beats Model B."
- **Directional accuracy** (% of weeks where you correctly predict the sign of price change) — relevant for procurement decisions

Also report **per-product** metrics, not just aggregated. Diesel and gasoline behave differently and your write-up should show that.

3.7. Feature importance / interpretability

For the final selected models, generate:

- **Coefficient paths** (for Ridge/Lasso) — visualize how β_brent_lag_k changes across k
- **Permutation importance** (`sklearn.inspection.permutation_importance`) — model-agnostic, works on linear and tree models, computed on a held-out fold to avoid bias

- **SHAP values** (`shap.TreeExplainer` for XGBoost) — per-prediction attributions, beautiful summary plots; the "How to understand high global food price" paper (PMC 2023) does exactly this for oil → food

This is critical for the "design constraints / explainability" rubric criterion. Black-box predictions without "why" lose points.

---

4. Specific gotchas you need to avoid (and how to demonstrate you avoided them in writing)

4.1. The "previous price as feature" trap

Eliwa et al. (2024) go from R^2=0.5620 to R^2=0.9970 just by adding `previous_price` as a feature. **This is not real predictive power.** Pump prices have weekly autocorrelation ~0.95 — including the previous week's price as a feature means your model is mostly outputting "next week ≈ this week," which is exactly the persistence baseline. **Always report your model's improvement over persistence, not just R^2/MAPE in isolation.** This is the MASE metric's whole purpose.

4.2. Look-ahead leakage from MOPS

The pricing formula uses *last week's* MOPS. If you accidentally include this week's MOPS as a feature, you're leaking information that wouldn't be available in real-time. The DOE announcement is Monday, prices change Tuesday, so for forecasting *next week's* price (week T+1), you have MOPS data complete through Friday of week T. This is fine. For multi-step (T+4), you must NOT use MOPS from weeks T+1, T+2, T+3 — those are unobserved at forecast time. Either:

- Use only lagged MOPS available at forecast origin (recursive or direct multi-step)
- Or use a multi-output model trained to produce all 4 horizons jointly

4.3. The structural-break problem (TRAIN excise hikes)

Pump prices have step changes from excise hikes Jan 2018, 2019, 2020. A naive autoregressive model trained across these dates will mis-attribute the steps to noise. Either:

- Include `excise_diesel_php_per_L` as a feature so the model can absorb the step
- Or model **price net of excise + VAT** as the target, then add taxes back at the end. This is cleaner economically.

4.4. Don't double-count MOPS and Brent

MOPS and Brent are >0.9 correlated. Don't drop both into a Ridge with no L2 — coefficients will

be unstable. Either:

- Use only one (MOPS preferred per §1)
- Or use both and rely on L2/L1 to handle collinearity (this is fine)
- Or use Brent and the *crack spread* (MOPS − Brent) — these are roughly orthogonal and give you both pieces of information cleanly

4.5. The "Rockets and Feathers Hypothesis" (RFH) — your novelty hook

The Wen et al. (2025) paper on your reading list is specifically about this. The headline finding:

- Gasoline prices respond *asymmetrically* to crude oil moves: rises pass through fast and fully, drops pass through slow and partially.
- Their predictive framework: split crude oil returns into `r⁺ = max(r, 0)` and `r⁻ = min(r, 0)` and use them as separate features. Out-of-sample R^2 jumps from negative (raw returns model loses to no-change baseline) to ~3-4% positive (asymmetric model).

You should test this on PH data. It's a clean, well-cited methodological extension that gives your project genuine novelty. Add `brent_pos_lag_k = max(brent_ret_lag_k, 0)` and `brent_neg_lag_k = min(brent_ret_lag_k, 0)` as features, then test whether this beats the symmetric specification with Diebold–Mariano. This also directly corresponds to the asymmetric DOE pricing rule (minimum rollback enforcement vs. unbounded hikes).

This is the move that makes your project stand out. PH-specific rocket-and-feather has very little prior literature; you'd be filling a real gap.

---

5. Pipeline architecture — directory layout and module organization

This addresses your prof's MP1 deductions on code organization (modules-by-source instead of by-functionality, `utils.py` doing too much, hard to find things).

```
pump_price_forecasting/
├── data/
|   ├── raw/                    # Untouched original pulls
|   |   ├── eia/                # EIA JSON dumps (semi-structured rubric
requirement)
|   |   ├── doe/                # DOE PDFs
|   |   ├── bsp/                # BSP HTML snapshots
|   |   ├── wb/                 # World Bank Pink Sheet XLSX
|   |   └── platts_proxy/       # Singapore gasoil/mogas via investing.com
|   ├── interim/                # Per-source cleaned (one Parquet per source)
|   └── final/                  # Joined analysis-ready dataset (Parquet + CSV)
├── src/
|   ├── ingestion/              # ALL data acquisition lives here
|   |   ├── eia_api.py          # API client
|   |   ├── doe_scraper.py      # PDF scraper, extended to 2017
|   |   ├── bsp_scraper.py      # BSP exchange rate scraper
|   |   ├── wb_pinksheet.py     # World Bank API/download
|   |   └── platts_proxy.py     # Singapore product prices
|   ├── cleaning/               # Source-agnostic cleaning utilities
|   |   ├── time_alignment.py   # ISO-week alignment, calendar handling
|   |   ├── outliers.py         # Winsorization, anomaly detection
|   |   └── imputation.py       # Forward-fill, median imputation
|   ├── features/               # Feature engineering
|   |   ├── lags.py             # Lag generation
|   |   ├── rolling.py          # Rolling stats
|   |   ├── derived.py          # crack_spread, MOPS_in_PHP, etc.
|   |   ├── calendar.py         # Holidays, week-of-year
|   |   └── events.py           # COVID, war, excise dummies
|   ├── splits/                 # Cross-validation
|   |   └── time_series_cv.py   # Walk-forward CV with gap
|   ├── models/                 # Model factories
|   |   ├── baselines.py        # Naive, seasonal naive, formula-based
|   |   ├── linear.py           # OLS, Ridge, Lasso, ElasticNet
|   |   ├── kernel.py           # SVR
|   |   ├── trees.py            # RF, GBM, XGBoost, LightGBM
|   |   └── classification.py   # Logistic, SVM-classifier, RF-classifier for
direction task
|   ├── evaluation/             # Metrics + tests
|   |   ├── metrics.py          # MAE, RMSE, MAPE, MASE, directional accuracy
|   |   ├── tests.py            # Diebold-Mariano
|   |   └── interpretability.py # Permutation importance, SHAP wrappers
|   ├── viz/                    # Visualization (DVS course)
|   |   ├── eda_plots.py
```

```
|   |   ├── model_plots.py       # Forecast vs actual, residuals, feature
importance
|   |   └── reporting.py         # Final report figures
|   └── pipelines/               # Orchestration
|       ├── build_dataset.py     # Run ingestion + cleaning + feature engineering
|       └── run_experiments.py   # Train all models, log metrics
├── tests/
|   ├── test_ingestion.py        # Mock HTTP with `responses`; fixes MP1 deduction
|   ├── test_cleaning.py
|   ├── test_features.py         # Verify no leakage in lag construction
|   ├── test_splits.py           # Verify CV splits are temporal & non-overlapping
|   └── test_models.py
├── notebooks/
|   ├── 01_eda.ipynb             # EDA-FIRST (fixes MP1 deduction); informs
feature choices
|   ├── 02_baselines.ipynb       # Tier 0
|   ├── 03_linear.ipynb          # Tier 1
|   ├── 04_kernel_trees.ipynb    # Tier 2
|   ├── 05_advanced_optional.ipynb # Tier 3
|   ├── 06_classification.ipynb  # Direction prediction
|   ├── 07_interpretability.ipynb # SHAP, permutation importance
|   └── 08_final_report.ipynb    # Polished narrative for submission
├── sql/
|   ├── schema.sql               # CREATE TABLE statements
|   └── queries/                 # Analytical queries used in modeling notebooks
|       ├── pump_x_brent_join.sql
|       └── coverage_report.sql
├── environment.yml
├── README.md
└── run_pipeline.py              # Top-level orchestration entry point
```

Why this is better than MP1's structure: organized by *function*, not by *source*. When you need to find "the function that builds Brent lag features," you go to `features/lags.py`, not `brent_pipeline.py`. Searching is intuitive. Same scrape boilerplate isn't duplicated across source files — it lives in `ingestion/` with a shared base class.

---

6. SQL schema — addressing the "SQL only used for storage, not analysis" deduction

```sql
```

```sql
-- pump_prices.db schema

CREATE TABLE oil_benchmarks (
    date DATE NOT NULL,
    benchmark TEXT NOT NULL,  -- 'brent' | 'wti' | 'dubai' | 'mops_gasoil' | 'mops_m
    price_usd_bbl REAL,
    log_return REAL,
    PRIMARY KEY (date, benchmark)
);
CREATE INDEX idx_oil_date ON oil_benchmarks(date);

CREATE TABLE fx_rates (
    date DATE PRIMARY KEY,
    usd_php REAL NOT NULL,
    usd_php_log_return REAL
);

CREATE TABLE pump_prices_station (
    monitoring_date DATE NOT NULL,
    effectivity_date DATE NOT NULL,
    city TEXT NOT NULL,
    product TEXT NOT NULL,        -- 'diesel' | 'ron91' | etc.
    brand TEXT,
    price_low REAL,
    price_high REAL,
    price_mid REAL,
    PRIMARY KEY (effectivity_date, city, product, brand)
);
CREATE INDEX idx_pp_date ON pump_prices_station(effectivity_date);
CREATE INDEX idx_pp_product ON pump_prices_station(product);

CREATE TABLE pump_prices_weekly (
    week_start DATE NOT NULL,
    product TEXT NOT NULL,
    price_median REAL,
    price_iqr REAL,
    n_stations INTEGER,
    PRIMARY KEY (week_start, product)
);

CREATE TABLE excise_taxes (
    effective_date DATE NOT NULL,
    product TEXT NOT NULL,
```

```sql
    excise_php_per_liter REAL NOT NULL,
    note TEXT,
    PRIMARY KEY (effective_date, product)
);
-- Pre-populated from TRAIN law schedule + RA 12316 suspensions

CREATE TABLE events (
    start_date DATE NOT NULL,
    end_date DATE,
    event_name TEXT NOT NULL,    -- 'covid_lockdown' | 'russia_ukraine_war' | 'opec_
    note TEXT,
    PRIMARY KEY (start_date, event_name)
);

CREATE TABLE feature_matrix (
    week_start DATE NOT NULL,
    product TEXT NOT NULL,
    -- ... all engineered features as columns
    PRIMARY KEY (week_start, product)
);
```

SQL queries actually used in analysis (not just storage):

- `pump_x_brent_join.sql`: joins weekly pump prices to lagged Brent + FX with proper week alignment
- `coverage_report.sql`: per-product, per-week obs counts and missingness — used in EDA
- `excise_history.sql`: returns the active excise rate at any given date (used in feature engineering)

Use these queries via `pd.read_sql_query()` in the modeling notebooks. That demonstrates SQL is doing real analytical work, not just storing data. This directly addresses your prof's MP1 comment.

---

7. Reading list / citations — what to cite where

You uploaded 5 papers. Here's exactly where each fits in your write-up:

Paper	Cite where
Lunor et al. (2023) "Machine Learning Approach for Pump Price Prediction for the Philippines Post COVID-19 Pandemic and Amidst Russia-Ukraine Conflict"	**The single most important paper for you.** It's the direct predecessor — same country, same products, same approach. Cite extensively in: §Introduction (motivation), §Methodology (PCA + MLR/SVR/RFR is their template), §Results (compare your MAPE 3-13% range to theirs), §Discussion (extend their work by adding 2017–2020 data, MOPS as a feature, RFH/asymmetry, and direction classification). **Frame your project as "Lunor et al. extended."**
Wen et al. (2025) "Forecasting gasoline prices using oil prices: New evidence based on the rocket and feather hypothesis"	Cite for: §Methodology (asymmetric features r^+ and r^-, threshold regression), §Results (compare your asymmetric vs symmetric finding), §Discussion (their economic explanation). This is your **novelty hook** — apply RFH to PH data.
He (2023) "Forecasting Gasoline Price with time Series Models"	Cite for: §Introduction (univariate vs multivariate), §Methodology (the LM1 specification with trend + seasonality + macro + crude — your linear baseline mirrors this), §Discussion (his finding that crude alone is not enough — supports your push for MOPS + FX + excise)
Eliwa et al. (2024) "Optimal Gasoline Price Predictions: Leveraging the ANFIS Regression Model"	Cite for: §Methodology (ANFIS is one of your "advanced/optional" tier 3 models), §Discussion (caution about previous-price-as-feature inflating R^2 — see §4.1 of this doc), §Comparison-of-models (their Table 9 directly: linear regression beat all ensemble methods on US gasoline; relevant comparator)
Ljubić et al. (2023) "Exploratory Data Analysis and Prediction of Fuel Prices in FBiH Relative to Oil Price Movements"	Cite for: §EDA section (their cross-canton scatter matrix + violin plots + Dickey-Fuller stationarity testing is a clean template you can mirror per-product or per-city), §Methodology (LSTM on differenced data; their data-shifting experiment for cross-correlation lag identification — do this with your data and you'll likely find PH lag is ~1 week vs their 32 days for Bosnia), §Discussion (LSTM as one of your optional advanced comparisons)

Additional references you should pull in:

- **Borenstein, Cameron & Gilbert (1997)** "Do gasoline prices respond asymmetrically to crude oil price changes?" — the original asymmetric pass-through reference (Quarterly Journal of Economics)
- **Bacon (1991)** "Rockets and feathers" — the original RFH paper

- **Bergmeir & Benítez (2012)** "On the use of cross-validation for time series predictor evaluation" — your justification for walk-forward CV
- **Diebold & Mariano (1995)** — your forecast-comparison test
- **Lu et al. (2021)** "Energy price prediction using data-driven models: A decade review" — your benchmark MAPE range (0.131%–19.2%)
- **Asche, Gjølberg & Völker (2003)** "Price relationships in the petroleum market" — Brent / refined product cointegration (justifies your crack-spread feature)
- **Kpodar & Abdallah (2017)** "Dynamic fuel price pass-through" — global retail fuel price database; methodology for measuring pass-through
- **Global Petroleum Prices (2016)** "Philippines Fuel Prices Outlook" — the Brent + FX + seasonal regression that explains 95%+ of PH retail fuel; direct PH-specific precedent

---

8. Sampling and frequency — concrete decisions

8.1. Sample period

- **Start: January 1, 2018** (post-TRAIN tranche 1; consistent excise structure)
  - If 2017 DOE data is recoverable in <2 days, push back to **Jan 2017**
- **End: most recent complete week** (e.g., last Friday before submission)
- **~430 weekly observations** at 2018 start; ~470 at 2017 start

8.2. Frequency

- **Modeling cadence: WEEKLY**, anchored to ISO Monday weeks.
- Daily Brent / WTI / FX → aggregated to weekly OHLC + weekly mean. Use **Friday close** as the canonical weekly value (matches the DOE Monday announcement which uses last week's averages).
- DOE pump prices → already weekly (Tuesday effectivity); align to Monday-of-the-week.
- World Bank Pink Sheet (Dubai monthly) → forward-fill to weekly; document the assumption.
- CPI/macro (monthly) → forward-fill to weekly OR linearly interpolate; Lunor et al. tested both and found similar performance, so document either choice.

8.3. Per-product or pooled?

You have 7 pump products. Two routes:

- **Per-product models (recommended for primary)**: one model per product. Total 7 models. Each is small but interpretable, and diesel vs gasoline genuinely differ (different MOPS benchmark, different excise tax).
- **Pooled "global" model**: one model with `product` as a categorical feature. Larger sample (7 × 430 = 3,000 rows). Worth doing as a comparison; if it beats per-product, you've shown that cross-product information helps.

Do **per-product as primary** (cleaner, easier to explain), then add pooled as a comparison and discuss.

---

9. Concrete plan — week-by-week (you have ~3 weeks until May 7)

Week	Days	Tasks
Week 1	Day 1	Repo restructure per §5; environment.yml; README skeleton
	Day 2	Extend DOE PDF scraper to 2017–2020; verify; persist to interim Parquet
	Day 3	Build EIA Brent/WTI ingestion; build BSP USD/PHP scraper; persist raw JSON + cleaned Parquet
	Day 4	Build World Bank Pink Sheet ingestion (Dubai monthly); build Singapore MOPS-proxy scraper
	Day 5	Hand-encode `excise_taxes` table from TRAIN schedule + RA 12316; encode events table; populate SQLite
	Days 6–7	Unit tests for all ingestion (mock HTTP via `responses`); `test_features.py` to verify zero leakage
Week 2	Day 1	Notebook 01 — EDA: stationarity, ACF/PACF, CCF Brent→pump, distribution plots, change-frequency analysis. **EDA-first justifies everything downstream.**
	Day 2	Notebook 02 — Tier 0 baselines (naive, seasonal naive, formula-based); set the bar
	Day 3	Feature engineering pipeline (lags, rolling, derived, calendar, events) → save feature matrix to `data/final/`
	Day 4	Notebook 03 — Tier 1 (Ridge, Lasso, ElasticNet) with walk-forward CV, hyperparameter tuning

Week	Days	Tasks
	Day 5	Notebook 04 — Tier 2 (SVR, RF, GBM, XGBoost) — hyperparameter tuning, comparison table
	Day 6	Apply PCA on lag-feature block; rerun best models on PCA-reduced features; document variance explained
	Day 7	Asymmetric / RFH analysis — split returns into r^+/r^-, retrain best linear & XGBoost models, Diebold–Mariano test
Week 3	Day 1	Notebook 06 — direction classification (logistic, SVM, RF classifier on 3-class hike/hold/rollback)
	Day 2	Notebook 07 — interpretability: permutation importance + SHAP for final XGBoost
	Day 3	Notebook 08 — final report: figures, decision log, comparison tables, residual diagnostics
	Day 4	Polish: PEP8 (`ruff check .`), docstrings, README, env.yml lockfile
	Day 5	Buffer: re-run end-to-end pipeline from raw to report; verify reproducibility
	Day 6	Final read-through; PR develop → main
	Day 7	Submission day (May 7)

10. What "good" looks like for the colloquium

By the end:

- **MAE on 1-week-ahead diesel forecast: target < ₱1.50/L** (≈2% MAPE). This would beat the persistence baseline meaningfully.
- **MAPE range across products: 2-5%**. This puts you firmly inside the Lu et al. (2021) benchmark range (0.131%–19.2%) and competitive with Lunor et al. (3.13%–12.67%).
- **Best model: probably Ridge or LightGBM with MOPS_in_PHP as the dominant feature**. If your top SHAP feature is anything other than `MOPS_in_PHP_lag1` (or its proxy `Brent_lag1 × USD_PHP_lag1`), something is wrong.
- **Asymmetric model significantly beats symmetric** (DM test $p < 0.05$) for at least gasoline products — replicating Wen et al.'s finding on PH data is a clean novelty contribution.

- **Direction classification accuracy: target > 70%** on the 3-class hike/hold/rollback task (vs ~33% random).
- **Clear visualizations**: forecast vs actual line plot for held-out test period; residual distribution; SHAP summary plot; feature-importance bar; confusion matrix for direction task.

---

11. Final honest note

The **honest reality** of this project: with the right features (MOPS + FX + lags + excise dummies), even simple Ridge regression should give you ~2-3% MAPE because the underlying mechanism is essentially linear and well-defined. Your contribution isn't going to be "we beat the state-of-the-art by 0.5% MAPE" — it's:

1. **A rigorous, reproducible, well-engineered pipeline for a real, locally-relevant problem** (PH pump prices, 2018–2026, all 7 products)
2. **A literature-grounded methodology** that mirrors what the published precedents (Lunor et al., He, Wen et al., Ljubić et al., Eliwa et al.) actually did
3. **An asymmetry / RFH analysis** that hasn't been published for PH specifically — your novelty hook
4. **A direction-classification side analysis** that bridges into ML Notebook 3B and gives the project decision-relevance
5. **An honest comparison** of model tiers (naive → linear → tree → optional advanced) that shows you understand model selection isn't "use the fanciest thing"

That's a strong colloquium project for three courses simultaneously. It's defensible, reproducible, hits every rubric, and tells a coherent story.

Good luck. Ping back once you have the EDA notebook done — it's the load-bearing wall for everything after.